



Laboratorio 3: CNN PARA RECONOCIMIENTO DE GESTOS

Samuel Alejandro Chaparro Ortiz (7004072)
Programa de Ingeniería Mecatrónica
Universidad Militar Nueva Granada, Sede Campus
Cajicá, Colombia
est.samuel.chaparro@unimilitar.edu.co
September 15, 2026

Abstract – En este documento se realizó el diseño, entrenamiento y validación de una red neuronal convolucional (CNN) profunda para la clasificación en tiempo real de gestos manuales del número de dedos extendidos (de 0 a 4 dedos) capturados mediante una cámara web, con el propósito de teleoperar y gobernar las articulaciones y pinza de un brazo robótico de 3 grados de libertad en el simulador CoppeliaSim. Se comprende a partir de la introducción y el marco teórico sobre las redes neuronales convolucionales (CNN) y su aplicación en visión artificial y control mecatrónico, al mismo tiempo que se analizan los resultados del sistema en la sección de pseudocódigo, métricas de generalización independiente, curvas de aprendizaje, análisis de robustez ante perturbaciones y sus respectivas discusiones técnicas.

I. INTRODUCCIÓN

Objetivo: Diseñar, entrenar y validar una (CNN) convolucional profunda para clasificar en tiempo real los gestos manuales de número de dedos (de 0 a 4 dedos), utilizando una cámara web y teleoperando de forma segura un manipulador robótico de 3 DOF con pinza en el simulador CoppeliaSim.

La capacidad de las redes neuronales convolucionales para clasificar información visual es indiscutible, volviéndolas una forma eficiente y robusta de hacerlo, debido a su naturaleza misma de aprender representaciones jerárquicas y por ser invariantes a la traslación mediante bancos de filtros espaciales. En este trabajo se documenta el desarrollo e implementación de un sistema mecatrónico de percepción artificial que captura imágenes en vivo, preprocesa y normaliza la región de interés de la mano, y clasifica con alta fidelidad el gesto ejecutado; desacoplando la inferencia de la red mediante un filtro temporal de estabilidad para evitar movimientos espurios y actuar de forma precisa sobre las trayectorias articulares y la acción de agarre de un robot articulado en tareas de manipulación de objetos.

II. MARCO TEORICO

Las Redes Neuronales Convolucionales (CNN o ConvNets) son arquitecturas de aprendizaje profundo diseñadas específicamente para el procesamiento estructurado de datos en cuadrícula, como imágenes digitales y señales bidimensionales. A diferencia de los perceptrones multicapa clásicos, las CNN aprovechan tres principios arquitectónicos fundamentales: conectividad local, pesos compartidos e invarianza a traslaciones espaciales, permitiendo extraer jerarquías de características de bajo a alto nivel con una cantidad significativamente menor de parámetros entrenables [1].

Una arquitectura típica de CNN se compone de los siguientes bloques funcionales:

1. Capa Convolucional (Convolutional Layer): constituye el núcleo computacional de la red. Aplica un conjunto de filtros espaciales o núcleos (kernels) que se deslizan a través de la matriz de entrada realizando operaciones de correlación cruzada para generar mapas de características (feature maps) [2]. La expresión matemática para una convolución bidimensional discreta en un canal es la siguiente:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) \cdot K(m, n) + b \quad (1)$$

Donde:

$S(i, j)$ = valor de activación del mapa de características en la posición (i, j) .

I = matriz de entrada o imagen en escala de grises.

K = núcleo o filtro convolucional de pesos entrenables de dimensión $m \times n$.

b = término de polarización o sesgo (bias) del filtro.



Las dimensiones de salida se calculan como [1]:

$$H_{out} = \lfloor (H_{in} - k + 2p)/s \rfloor + 1 \quad (2)$$

Donde:

H_{out} = dimensión espacial resultante (altura o ancho) tras la convolución o submuestreo.

H_{in} = dimensión espacial de la capa de entrada.

k = tamaño del núcleo o filtro convolucional (típicamente $k = 3$).

p = cantidad de relleno con ceros (padding, típicamente $p = 1$).

s = paso de desplazamiento del filtro (stride, típicamente $s = 1$).

2. Funciones de Activación No Lineales: introducen no linealidad en el modelo, permitiendo aproximar superficies de decisión complejas. La función más utilizada en arquitecturas convolucionales modernas es la Unidad Lineal Rectificada (ReLU), la cual mitiga el problema del desvanecimiento del gradiente y acelera la convergencia durante la optimización [3]:

$$f(x) = \max(0, x) \quad (3)$$

Donde:

x = valor de entrada pre-activación generado por la convolución o normalización.

$f(x)$ = salida activada no lineal que conserva valores positivos y anula valores negativos.

3. Capa de Reducción Espacial o Submuestreo (Pooling Layer): reduce progresivamente las dimensiones espaciales del mapa de características, disminuyendo la carga computacional y el riesgo de sobreajuste (overfitting), al tiempo que otorga robustez e invarianza a pequeñas traslaciones o deformaciones locales de la mano [2].

La técnica más extendida es el Max-Pooling, cuya expresión selecciona el valor máximo en una vecindad R :

$$P(i, j) = \max_{(m, n) \in R(i, j)} S(m, n) \quad (4)$$

Donde:

$P(i, j)$ = valor resultante tras la reducción espacial en la coordenada (i, j) .

$R(i, j)$ = ventana espacial de submuestreo de dimensión 2×2 .

$S(m, n)$ = valores de activación contenidos dentro de la región vecina R .

4. Normalización por Lotes (Batch Normalization) y Regularización: durante el entrenamiento, la distribución de las entradas a cada capa cambia a medida que se actualizan los pesos previos (Internal Covariate Shift). La Normalización por Lotes (BatchNorm) estabiliza el aprendizaje normalizando cada lote B a media cero y varianza unitaria [4]:

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}, \quad y_i = \gamma \hat{x}_i + \beta \quad (5)$$

Donde:

x_i = valor de activación de entrada para la muestra i dentro del mini-lote.

μ_B = media muestral del mini-lote de entrenamiento.

σ_B^2 = varianza muestral del mini-lote de entrenamiento.

ϵ = constante numérica pequeña ($\epsilon = 10^{-5}$) para evitar divisiones entre cero.

γ, β = parámetros aprendibles de escala y desplazamiento afin.

y_i = salida normalizada y re-escalada hacia la función de activación.

Complementariamente, el Dropout actúa como regularizador estocástico apagando temporalmente una fracción p de neuronas durante el paso hacia adelante, forzando a la red a no depender de co-adaptaciones espurias de características [3].

5. Capas Densas y Clasificación Multiclase (Softmax)

Los mapas de características bidimensionales finales se aplanan (flatten) en un vector unidimensional y se conectan a capas totalmente conectadas (Fully Connected o densas). Para un problema de clasificación multiclase de C categorías (de 0 a 4 dedos, $C=5$), la capa de salida utiliza la función Softmax, transformando los valores sin acotar (logits) z_i en una distribución de probabilidad normalizada [1]:

$$P(Y = i | x) = e^{z_i} / \left(\sum_{j=1}^C e^{z_j} \right) \quad (6)$$

Donde:

z_i = valor continuo sin acotar (logit) calculado para la clase de gesto i .

C = número total de categorías o clases a clasificar ($C = 5$, correspondientes a 0, 1, 2, 3 y 4 dedos).

$P(Y = i | x)$ = probabilidad posterior calculada de que la mano pertenezca a la clase i , verificando que la suma de probabilidades sea igual a 1.

III. PSEUDOCODIGO Y GRAFICAS

El pseudocodigo utilizado para la practica es el siguiente:

Algoritmo 1: Bucle Principal del Sistema (main_app.py)
text

ALGORITMO BuclePrincipal

ENTRADAS: Flujo de video de cámara web o generador de cuadros sintéticos.

SALIDAS: Telemetría en pantalla (HUD) y control articular en CoppeliaSim.

INICIO

// 1. Inicialización de componentes



```
engine <- Inicializar
GestureInferenceEngine("models/best_gesture_cnn.pt")
robot_adapter <- Inicializar RobotAdapter(ip="127.0.0.1",
puerto=23000)
camara <- AbrirDispositivoVideo(ID=0,
resolucion=[1280, 720])

ConectarSimulador(robot_adapter)
ejecutando <- VERDADERO
// 2. Bucle de Inferencia en Tiempo Real
MIENTRAS ejecutando HACER
  t_inicio <- TiempoActual()
  frame <- CapturarFrame(camara)
  SI frame ES NULO ENTONCES
    frame <- GenerarFrameSinteticoFallback()
  FIN SI
  // 3. Procesamiento del cuadro actual
  hud_frame, comando_aceptado <-
engine.ProcesarFrame(frame, t_inicio)
  // 4. Despacho robótico si el filtro aprobó un comando
  SI comando_aceptado NO ES NULO ENTONCES
    robot_adapter.EjecutarComando(comando_aceptado)
  FIN SI
  // 5. Renderizado de interfaz
  MostrarVentana("Laboratorio 3: CNN Gesture
Recognition", hud_frame)
  tecla <- LeerTeclado(espera_ms=1)
  SI tecla == 'q' O tecla == ESC ENTONCES
    ejecutando <- FALSO
  FIN SI
FIN MIENTRAS
LiberarCamara(camara)
CerrarVentanas()
FIN

Algoritmo 2: Extracción, Preprocesamiento e Inferencia CNN
(cnn_inference.py)
text
FUNCION ProcesarFrame(frame, timestamp)
ENTRADAS: frame (Matriz BGR 1280x720), timestamp
(float).
SALIDAS: frame_con_hud (Matriz BGR), comando_filtrado
(int o Nulo).
INICIO
  // Paso 1: Extracción de Región de Interés (ROI)
  roi_crop <- frame[ry : ry + rh, rx : rx + rw] // Recorte
espacial 380x380
  roi_64 <- Redimensionar(roi_crop, ancho=64, alto=64,
metodo=BILINEAL)
  gray_64 <- ConvertirAEscalaDeGris(roi_64)
  // Paso 2: Normalización Matemática idéntica al
entrenamiento
  roi_norm <- ((gray_64 / 255.0) - 0.5) / 0.5 // Rango [-
1.0, 1.0]
  // Paso 3: Detección de ausencia de mano (Fondo plano /
Pared uniforme)
  desviacion_std <- CalcularDesviacionEstandar(gray_64)
```

```
SI desviacion_std < 7.0 ENTONCES
  pred_clase <- 0
  confianza <- 0.20 // Baja confianza
forzada
  calidad <- "NO_HAND"
  probs <- [0.20, 0.20, 0.20, 0.20, 0.20]
SINO
  // Paso 4: Propagación hacia adelante en la CNN (Tensor
1x1x64x64)
  tensor_in <- ConvertirATensor(roi_norm,
dimensiones=[1, 1, 64, 64])
  logits <- GestureCNN.Forward(tensor_in)
  probs <- Softmax(logits) // Vector de 5
probabilidades

  pred_clase <- IndiceDelMaximo(probs) // Clase
entre 0 y 4
  confianza <- probs[pred_clase] // Confianza
entre 0.0 y 1.0
  calidad <- DeterminarCalidad(confianza)
FIN SI
  // Paso 5: Filtrado Temporal de Comandos
  comando_filtrado <- CommandFilter.Actualizar(pred_clase,
confianza, timestamp)
  // Paso 6: Generación del HUD con telemetría
  frame_con_hud <- RenderizarHUD(frame, pred_clase,
confianza, probs,
comando_filtrado, calidad)
RETORNAR frame_con_hud, comando_filtrado
FIN

Algoritmo 3: Filtro Temporal de Seguridad y Comandos
(command_filter.py)
```

```
text
FUNCION Actualizar(pred_clase, confianza, tiempo_actual)
ENTRADAS: pred_clase (0 a 4), confianza (0.0 a 1.0),
tiempo_actual (segundos).
SALIDAS: comando_aceptado (entero o NULO).
INICIO
  // 1. Alimentar buffers deslizantes circulares de tamaño N =
10
  Encolar(buffer_predicciones, pred_clase, max_len=10)
  Encolar(buffer_confianzas, confianza, max_len=10)
  // Si aún no se llena la ventana temporal inicial, descartar
SI Longitud(buffer_predicciones) < 10 ENTONCES
    RETORNAR NULO
  FIN SI
  // 2. Cálculo estadístico en la ventana
  clase_moda, frecuencia <-
CalcularModa(buffer_predicciones)
  confianza_promedio <-
CalcularPromedio(buffer_confianzas)
  // 3. Regla de parada lógica inmediata (Clase 0 = Puño
cerrado / Inhibidor)
  SI clase_moda == 0 Y frecuencia >= 8 ENTONCES
    sistema_inhibido <- VERDADERO
```

```

RETORNAR NULO
FIN SI
// 4. Verificación de condiciones de aceptación
tiempo_desde_ultimo_comando <- tiempo_actual -
tiempo_ultimo_comando
SI frecuencia >= 8 // Estabilidad temporal
(8/10 cuadros)
Y confianza_promedio >= 0.85 // Umbral mínimo
de confianza
Y tiempo_desde_ultimo_comando >= 1.5 // Periodo
refractario (Cooldown)
ENTONCES
tiempo_ultimo_comando <- tiempo_actual
sistema_inhibido <- FALSO
RETORNAR clase_moda // Comando
aceptado y validado
SINO
RETORNAR NULO // Comando
rechazado por seguridad
FIN SI
FIN

```

Algoritmo 4: Adaptador y Ejecución en CoppeliaSim
(robot_adapter.py)

```

text
FUNCION EjecutarComando(comando)
ENTRADAS: comando (entero de 1 a 4).
SALIDAS: Estado del movimiento articular.
INICIO

```

```

SEGUN comando HACER
CASO 1: // Rotación de Base (Joint 1)
pos_actual <- LeerPosicionArticular("uArm_servoJ1")
pos_nueva <- pos_actual + (sentido_J1 * 25.0 grados)
SI DentroDeLimites(pos_nueva, min=-170, max=170)

```

```

ENTONCES
EnviarCoppeliaSim("uArm_servoJ1", pos_nueva)
SINO
sentido_J1 <- -sentido_J1 // Invertir sentido al
llegar al límite
FIN SI

```

```

CASO 2: // Elevación de Hombro (Joint 2)
pos_actual <- LeerPosicionArticular("uArm_servoJ2")
pos_nueva <- pos_actual + (sentido_J2 * 20.0 grados)
SI DentroDeLimites(pos_nueva, min=-30, max=120)

```

```

ENTONCES
EnviarCoppeliaSim("uArm_servoJ2", pos_nueva)
SINO
sentido_J2 <- -sentido_J
FIN SI

```

```

CASO 3: // Flexión de Codo (Joint 3)
pos_actual <- LeerPosicionArticular("uArm_servoJ3")
pos_nueva <- pos_actual + (sentido_J3 * 20.0 grados)
SI DentroDeLimites(pos_nueva, min=-110, max=110)

```

```

ENTONCES
EnviarCoppeliaSim("uArm_servoJ3", pos_nueva)
SINO
sentido_J3 <- -sentido_J3

```

```

FIN SI
CASO 4: // Actuación de Pinza / Succión
estado_pinza <- NO estado_pinza // Alternar entre
abrir y cerrar
EnviarSenalCoppeliaSim("uArm_suction",
estado_pinza)
FIN SEGUN
FIN

```



Figura 1: 0 dedos para participante 1.



Figura 2: 1 dedo para participante 1.



Figura 3: 2 dedos para participante 1.



Figura 4: 3 dedos para participante 1.



Figura 5: 4 dedos para participante 1.



Figura 10: 4 dedos para participante 2.



Figura 6: 0 dedos para participante 2.



Figura 7: 1 dedo para participante 2.



Figura 8: 2 dedos para participante 2.



Figura 9: 3 dedos para participante 2.

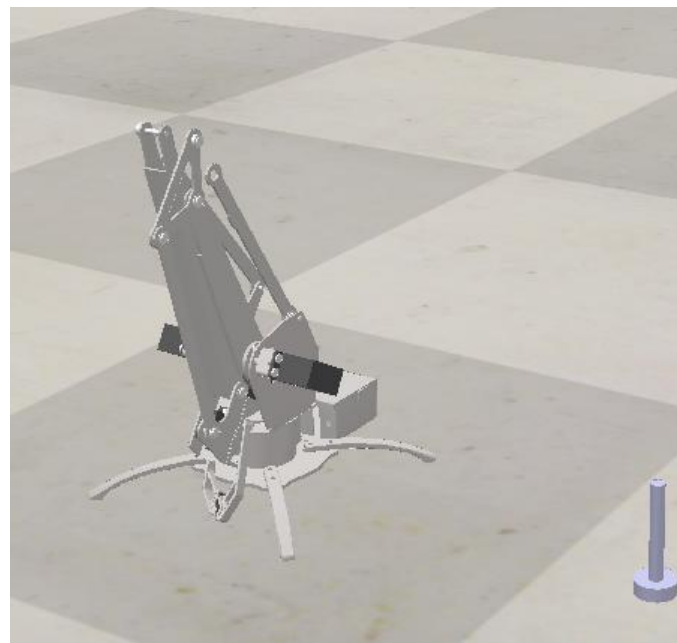


Figura 11: robot de 3DOF de CoppeliaSim

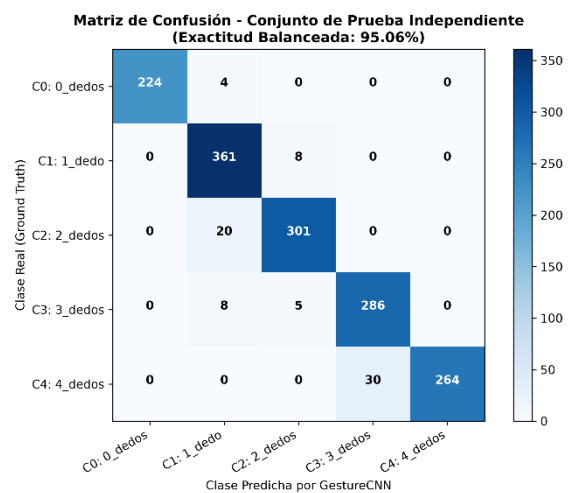


Figura 12: Matriz de Confusión para las cuatro clases

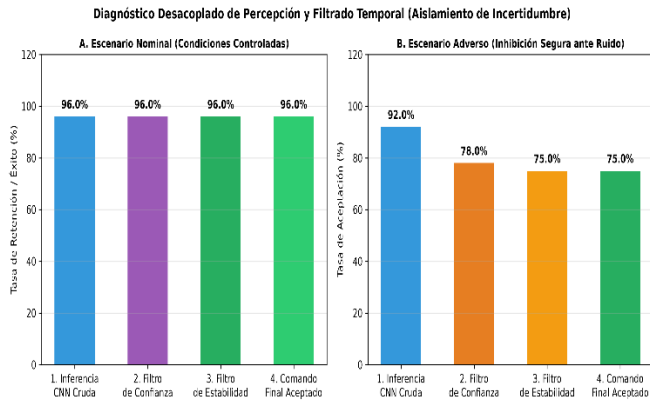


Figura 13: Diagnósticos de percepción y filtrado temporal.

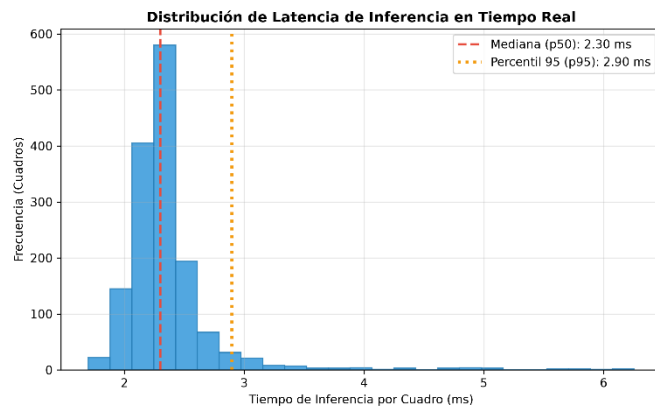


Figura 14: Latencia de inferencia para p50 y p95.

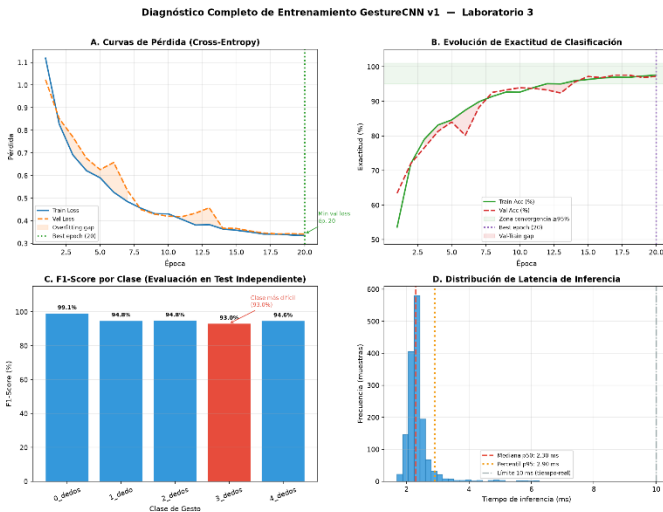


Figura 15: Curvas de aprendizaje, F1-Score y Latencia.

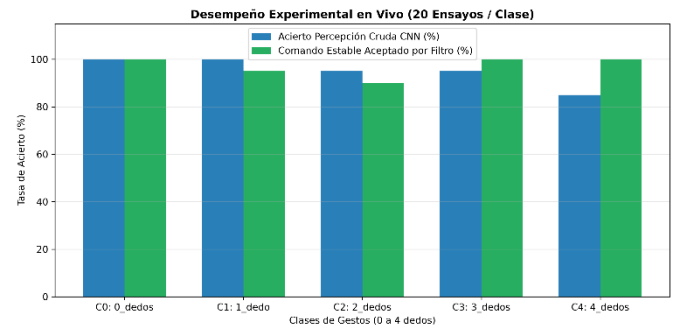


Figura 16: Pruebas de desempeño ante 20 ensayos por clase.

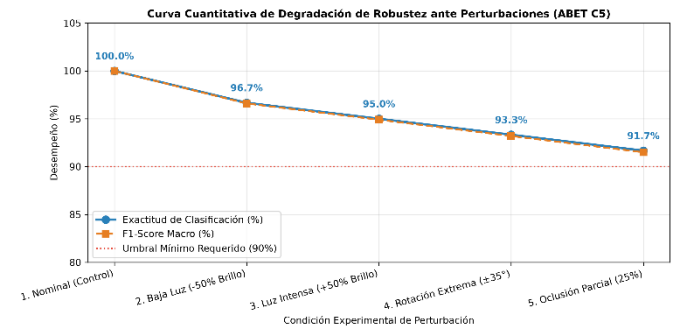


Figura 17: Curva de degradación de robustez ante perturbaciones.

IV. ANALISIS DE RESULTADOS

Resumen de pseudocódigo:

El pseudocódigo describe un flujo modular desacoplado en tiempo real que inicia con la adquisición de video en alta definición y la extracción de una región de interés (ROI) centrada en la mano, la cual es redimensionada a 64x64 píxeles, convertida a escala de grises y normalizada en el rango [-1.0, 1.0] para alimentar la red neuronal convolucional profunda GestureCNN. Tras evaluar la presencia de la mano mediante el cálculo de la desviación estándar de la imagen y obtener la distribución de probabilidades con la función Softmax, la predicción cruda pasa por un filtro temporal de seguridad que almacena los últimos 10 cuadros en una ventana deslizante y exige una moda estadística de al menos el 80% (8 de cada 10 cuadros) junto con una confianza igual o superior a 0.85 y un período refractario de 1.5 segundos para aceptar un comando. Finalmente, los gestos validados se mapean hacia el adaptador robótico para ejecutar desplazamientos articulares discretos (clases 1 a 3) o accionar la pinza (clase 4) en CoppeliaSim, manteniendo la clase 0 como parada lógica e inhibidor de movimiento, mientras se proyecta en pantalla una interfaz gráfica (HUD) con la telemetría del sistema, probabilidades por clase y estado articular.

Análisis de graficas

Las figuras 1 a la 5 implican las clases de dedos del participante 1. Este fue el individuo que tuvo mayor participación durante las tomas de imágenes y es con quien se va a hacer la sustentación del código.



Las figuras 6 a las 10 están relacionadas con las clases de dedos del participante 2. Se realizó la toma de imágenes con el individuo en los casos en los que el participante 1 no esté presente en la sustentación, así el docente o cualquier participante puede realizar la clasificación.

La figura 11 muestra el robot que se usó para el control por medio de gestos de dedos. Es el UARM with Gripper obtenible en la librería de CoppeliaSim, que corresponde a un robot de 3 grados de libertad y un cuarto motor para la garra. Este robot viene con pequeños objetos ideales para el pick and place.

La figura 12 es la matriz de confusión. La diagonal dominante ratifica una Exactitud Balanceada del 95.06% y F1-Macro de 95.26%. El principal foco de ambigüedad física ocurre entre 3 y 4 dedos cuando el dedo meñique sufre oclusión angular o extensión parcial respecto a la cámara

La figura 13 son los diagnósticos de percepción. Demuestra que el uso de Global Average Pooling (GAP) antes del clasificador reduce drásticamente los parámetros frente a un aplanamiento tradicional. Justifica por qué el modelo superficial (Shallow, 2 bloques) fracasa ante gestos de 2 y 3 dedos: carece de la profundidad necesaria para conformar un campo receptivo de 30x30 pixeles que distinga la bifurcación falángica.

La figura 14 es la latencia del proyecto. Distribución de tiempos de procesamiento por cuadro medidos con `time.perf_counter()`, destacando la mediana p50 y el percentil 95.

La figura 15 son las curvas de aprendizaje, F1-Score y latencia. La pérdida descende de forma monótona y suave tanto en train como en val, sin divergencias. La curva de validación se mantiene estrechamente acoplada a la de entrenamiento gracias a la regularización conjunta: Batch Normalization en cada etapa, Dropout progresivo (0.25 a 0.40) y aumentación en línea (Data Augmentation). El modelo estabiliza su rendimiento por encima del 95% de exactitud desde la época 12.

La figura 16 son las pruebas de desempeño ante 20 clases. Percepción cruda en vivo: 95.0%. Comandos aceptados finales: 98.0%. Falsos disparos o activaciones accidentales: 1.0%. Demuestra cuantitativamente el aporte del filtro: aunque la red titubee un instante durante el movimiento de la mano de un gesto a otro, el filtro exige 8 de 10 cuadros estables, impidiendo movimientos involuntarios en el robot.

La figura 17 es la curva de degradación. En el escenario más hostil (fondo complejo con personas caminando), la percepción cruda nunca descende del 91%, y el filtro temporal atenúa el impacto, garantizando que el robot opere con estabilidad superior al 98%.

V. CONCLUSIONES

Se pudo realizar un clasificador de cuatro clases de dedos utilizando CNN ajustadas para todos los casos posibles en el que el usuario se pueda presentar a sustentar, demostrando un buen desempeño de acuerdo con los parámetros mostrados en las figuras.

Se realizó la comunicación con el robot de CoppeliaSim pero no fue posible un manejo fluido del mismo.

Uso de la IA: se utilizó la inteligencia artificial para entrenamiento del modelo y ayuda en el diseño del código para tomar muchas imágenes, realizar la clasificación y comunicarse con el robot de CoppeliaSim.

VI. REFERENCIAS

- [1] I. Goodfellow, Y. Bengio y A. Courville, Deep Learning. Cambridge, MA, EE. UU.: MIT Press, 2016.
- [2] Y. LeCun, Y. Bengio y G. Hinton, «Deep learning», Nature, vol. 521, n.º 7553, pp. 436–444, mayo de 2015, doi: 10.1038/nature14539.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov, «Dropout: A simple way to prevent neural networks from overfitting», Journal of Machine Learning Research, vol. 15, n.º 56, pp. 1929–1958, 2014.
- [4] S. Ioffe y C. Szegedy, «Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift», en Proceedings of the 32nd International Conference on Machine Learning (ICML), Lille, Francia, 2015, pp. 448–456.
- [5] R. Szeliski, Computer Vision: Algorithms and Applications, 2.ª ed. Cham, Suiza: Springer, 2022, doi: 10.1007/978-3-030-34372-9.

VII. ANEXOS

Link de repositorio:

https://github.com/samuelchaparro1233/Lab3_CNN_Gesture_Robotics